PDE4444
Machine Learning for Engineers

Week 7

Types of Problems

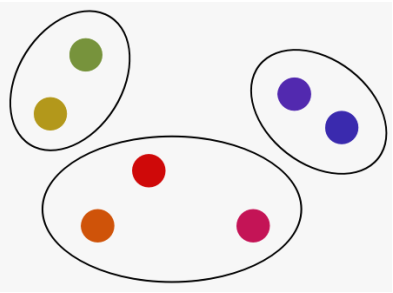
Supervised Learning

Regression

Classification

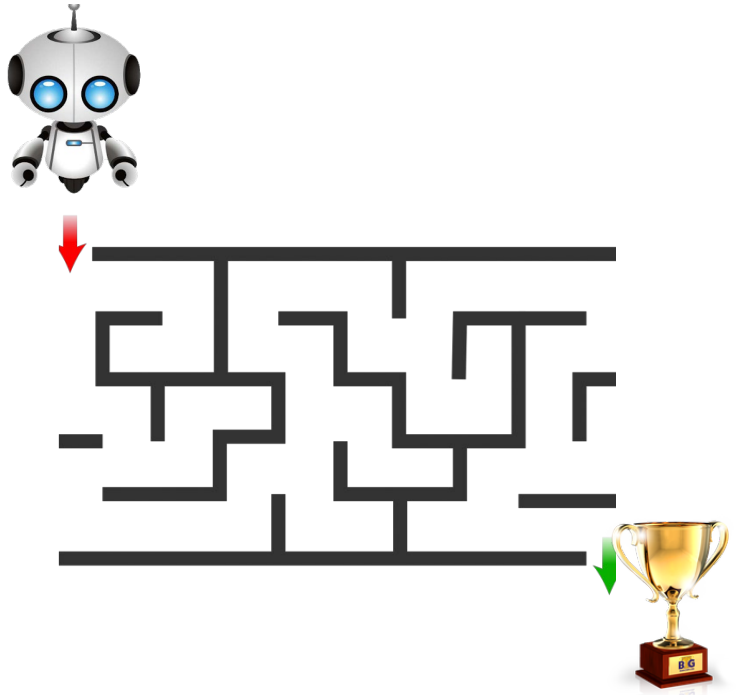
Unsupervised Learning

Association



Clustering

Reinforcement Learning



Types of data

Supervised Learning

Labelled Data

Cat

Algorithm

Dog

Cat

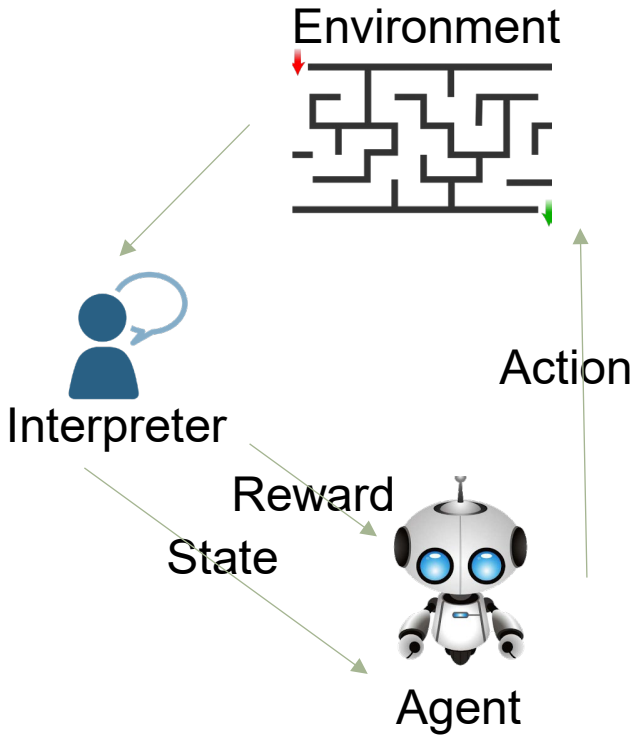
Unsupervised Learning

Unlabelled Data

Algorithm

Reinforcement Learning

No Predefined Data



Aim

Supervised Learning

Forecast Outcomes

Unsupervised Learning

Discover underlying patterns

Reinforcement Learning

Learn series of actions

Machine Learning Workflow

The general structure of a machine learning study/project stays relatively constant:

1. Data cleaning and formatting
2. Exploratory data analysis
3. Feature engineering and selection
4. Establish a baseline and compare several machine learning models on a performance metric
5. Perform hyperparameter tuning on the best model to optimize it for the problem
6. Evaluate the best model on the testing set
7. Interpret the model results to the extent possible
8. Draw conclusions and write a well-documented report

Programming with Datasets

Programming language

- Python!

Working with datasets

- Variables, lists, tuples, dictionaries, sets, etc. offer only temporary data storage
  - lost when a local variable “goes out of scope” or when the program terminates
- Data maintained in files is persistent

Working with datasets

- We consider text files in several popular formats
  - Plain text
  - JSON (JavaScript Object Notation)
  - CSV (comma-separated values)

- Use JSON to serialize and deserialize objects for saving to secondary storage and transmitting them over the Internet

- Loading and manipulating CSV data with Python Standard Library's `csv` module

Dataset with Text File

- **Built-in function** `open`

  Opens the file `accounts.txt` and associates it with a file object.

  `mode` argument specifies the **file-open mode**

    - whether to open a file for reading from the file, for writing to the file or both.

  - Mode `'w'` opens the file for *writing*, creating the file if it does not exist
  - If you do not specify a path to the file, Python creates it in the current folder
  - **Be careful**—opening a file for writing *deletes* all the existing data in the file
  - By convention, the **`.txt` file extension** indicates a plain text file

Dataset with Text File- Try it out

- **Writing to a Text File:**

```
with open('accounts.txt', mode='w') as accounts:
    accounts.write('100 Jones 24.98\n')
    accounts.write('200 Doe 345.67\n')
    accounts.write('300 White 0.00\n')
    accounts.write('400 Stone -42.16\n')
    accounts.write('500 Rich 224.62\n')
```

Can also **write to a file with `print`** which **automatically outputs a `\n`**, as in

```
print('600 Jones 24.98', file=accounts)
```

Dataset with Text File- Try it out

- **Reading data from a Text File**

```
with open('accounts.txt', mode='r') as accounts:
    print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
    for record in accounts:
        account, name, balance = record.split()
        print(f'{account:<10}{name:<10}{balance:>10}')
```

- Iterating through a file object, reads one line at a time from the file and returns it as a string
- For each `record` (that is, line) in the file, string method `split` returns tokens in the line as a list
  - We unpack into the variables `account`, `name` and `balance`

Dataset with JSON

JSON (JavaScript Object Notation) is a text-based, human-and-computer-readable, data-interchange format used to represent objects as collections of name–value pairs.

- Preferred data format for transmitting objects across platforms.

Dataset with JSON

JSON Data Format

- Similar to Python dictionaries
- Each **JSON object** contains a comma-separated list of **property names** and **values**, in curly braces.

```
{"account": 100, "name": "Jones", "balance": 24.98}
```

- **JSON arrays**, like Python lists, are comma-separated values in square brackets.

```
[100, 200, 300]
```

- Values in JSON objects and arrays can be:
  - strings in double quotes
  - numbers
  - JSON Boolean values `true` or `false`
  - `null` (like `None` in Python)
  - arrays
  - other JSON objects

Dataset with JSON

Python Standard Library for JSON: `json`

- **`json` module** enables you to convert objects to JSON (JavaScript Object Notation) text format
- Known as **serializing** the data
- Following dictionary, which contains one key–value pair consisting of the

key `'accounts'` with its associated value being a list of dictionaries representing two accounts

```
accounts_dict = {'accounts': [
   {'account': 100, 'name': 'Jones', 'balance': 24.98},
   {'account': 200, 'name': 'Doe', 'balance': 345.67}]}
```

Dataset with JSON- Try it out

Write JSON to a file:

- `json` module's **`dump` function** serializes the dictionary `accounts_dict` into the file

```
import json
with open('accounts.json', 'w') as accounts:
   json.dump(accounts_dict, accounts)
```

- Resulting file contains the following text—reformatted slightly for readability:

```
{"accounts":
[{"account": 100, "name": "Jones", "balance": 24.98},
 {"account": 200, "name": "Doe", "balance": 345.67}]}
```

- JSON delimits strings with *double-quote characters*.

Dataset with JSON- Try it out

- **Reading data from JSON file:**
  - `json` module's **`load` function** reads entire JSON contents of its file object argument and converts the JSON into a Python object
  - Known as **deserializing** the data

```
with open('accounts.json', 'r') as accounts:
    accounts_json = json.load(accounts)
```

  - Components accessed by:

```
accounts_json                          # show the full data as object
accounts_json['accounts']   # show all data rows
accounts_json['accounts'][0] # first row
accounts_json['accounts'][1]  # second row
```

Dataset with JSON- Try it out

- **Displaying JSON Text:**

```
with open('accounts.json', 'r') as accounts:
    print(json.dumps(json.load(accounts), indent=4))
```

Dataset with csv

- **CSV: Comma-Separated Values**, a plain text format for storing tabular data
- Python standard library module: `csv`
- **`csv` module** provides functions for working with CSV files

Dataset with csv- Try it out

Writing to a CSV File

- `csv` module's documentation recommends opening CSV files with the additional keyword argument `newline=''` to ensure that newlines are processed properly

```
import csv
with open('accounts.csv', mode='w', newline='') as accounts:
    writer = csv.writer(accounts)
    writer.writerow([100, 'Jones', 24.98])
    writer.writerow([200, 'Doe', 345.67])
    writer.writerow([300, 'White', 0.00])
    writer.writerow([400, 'Stone', -42.16])
    writer.writerow([500, 'Rich', 224.62])
```

Dataset with csv- Try it out

Reading a CSV File

```
with open('accounts.csv', 'r', newline='') as accounts:
    print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
    reader = csv.reader(accounts)
    for record in reader:
        account, name, balance = record
        print(f'{account:<10}{name:<10}{balance:>10}')
```

Data Preprocessing

Working with More Complex Data Sets

- Many real-world data sets are complex, structured as two-dimensional tables with multiple rows and columns
- These structured data can be saved in a special type of text file in **CSV format** for export to other data analysis programs
- This section explores the use of Python's `pandas` library to load data sets from CSV files and clean them before performing analyses and visualizations

What is `pandas` library

- `Pandas` is a Python library for **data analysis and manipulation**.
- Provides two key data structures:
  - **Series**: 1D labeled array.
  - **DataFrame**: 2D labeled table, similar to a spreadsheet.
- **Why Use `Pandas`?**
  - Efficient handling of large datasets like IoT sensor logs.
  - Easy integration with **Matplotlib** and **Seaborn** for visualization.
  - Built-in functions for data cleaning, filtering, and summarization.

The Dataset

Objective: Extend visualization concepts to handle complex, real-world data stored in CSV files.

We are going to use the **smart_sensors.csv** file for this exercise. The file represents the entire dataset related to Smart Homes from Kaggle. Link to data set: https://www.kaggle.com/datasets/pythonafroz/smart-home-data-set

- **Dataset**: The large IoT dataset (smart_sensors.csv) contains over **600,000 rows** and tracks:
  - **Temperature (temp)** and **Humidity (humid)**
  - **Brightness (bright)**, **Distance (dist)**, **Sound Level (soundlevel)**
  - Timestamps (result_time) and Node IDs (nodeid)
- **Tools**: Use **Pandas** for data manipulation and **Seaborn**/Matplotlib for visualization.

Loading and Exploring the Dataset

```
import pandas as pd
# Load the dataset with low_memory=False
df = pd.read_csv('smart_sensors.csv', low_memory=False)

# Check the first few rows
print(df.head())

# Summary of the dataset
print(df.info())
```

Key Insights:

- Dataset contains over **600,000 rows**.
- Columns like `temp` and `humid` have mixed types (string and float).
- Use `low_memory=False` forces Pandas to read the file in one go.

Data Cleaning- Try it out

- **Objective**: Clean and prepare the dataset for analysis.
- **Steps**:
  - Convert temp and humid to numeric values.
  - Remove invalid rows (e.g., negative distance values).
  - Fill missing values with meaningful substitutes (e.g., column means).

```
# Convert temp and humid to numeric, replacing invalid values with NaN
df['temp'] = pd.to_numeric(df['temp'], errors='coerce')
df['humid'] = pd.to_numeric(df['humid'], errors='coerce')

# Remove invalid distance readings
df = df[df['dist'] > 0].copy()

# Fill missing humidity values with the mean
df['humid'] = df['humid'].fillna(df['humid'].mean())
df['temp'] = df['temp'].fillna(df['temp'].mean())

# Verify the cleaned data
print(df.describe())
```

Exploring DataFrames- Try it out

- **Objective**: Summarize and filter data to uncover insights.

```
# Summary statistics for numerical columns
print(df[['temp', 'humid', 'bright', 'dist']].describe())

# Filter rows where temperature exceeds 30°C
high_temp = df[df['temp'] > 30]

# Group by node ID and calculate average temperature
avg_temp_by_node = df.groupby('nodeid')['temp'].mean()
print(avg_temp_by_node)
```

Data Visualization

Data visualization

- A **data analysis** application attempts to discover patterns in data to provide support for decisions in many different personal and institutional settings.
- **Data visualization** generally involves the graphical representation of data in such a form as to assist in their analysis.

Pie Charts Example

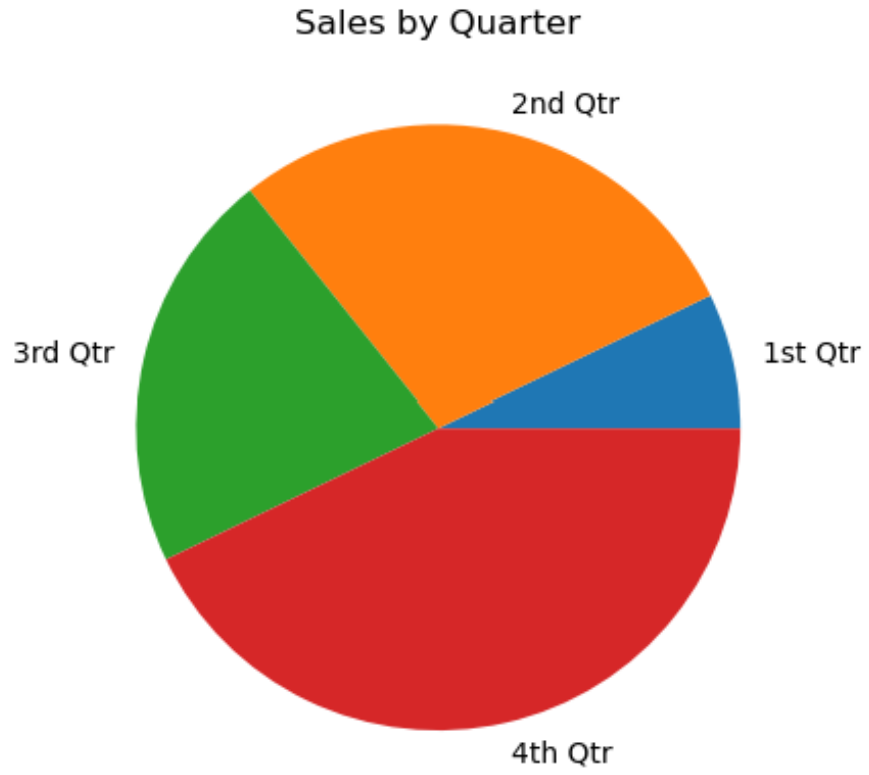
```
import matplotlib.pyplot as plt
values = [20, 60, 80, 40]
plt.pie(values)
plt.show()
```

Plotting a Pie Chart

- The `pie` function has a `labels` parameter that you can use to display labels for the slices in the pie chart.
- The argument that you pass into this parameter is a list containing the desired labels, as strings.

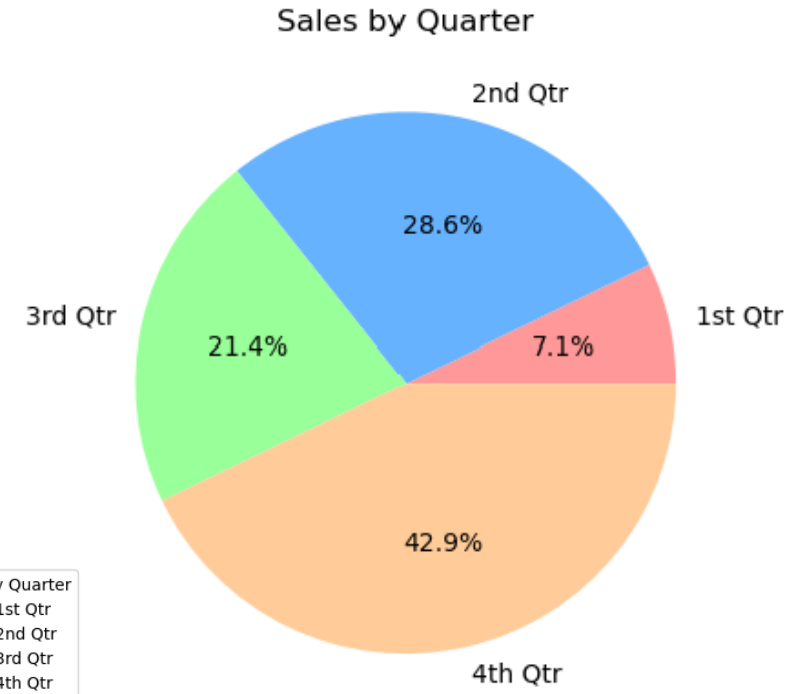
Pie Charts Example

```
sales = [100, 400, 300, 600]
slice_labels = ['1st Qtr', '2nd Qtr', '3rd Qtr', '4th Qtr']
plt.pie(sales, labels=slice_labels)
plt.title('Sales by Quarter')
plt.show()
```

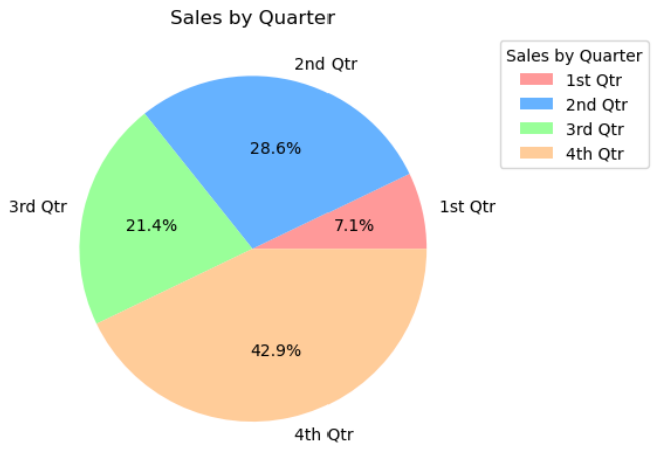


Pie Charts Example

```
sales = [100, 400, 300, 600]
slice_labels = ['1st Qtr', '2nd Qtr', '3rd Qtr',
'4th Qtr']
colors = ['#FF9999', '#66B2FF', '#99FF99',
'#FFCC99'] # Custom colors
plt.pie(sales, labels=slice_labels, autopct = "%1.1f%%",
colors=colors)
plt.title('Sales by Quarter')
plt.show()
```



Explore adding a Legend:



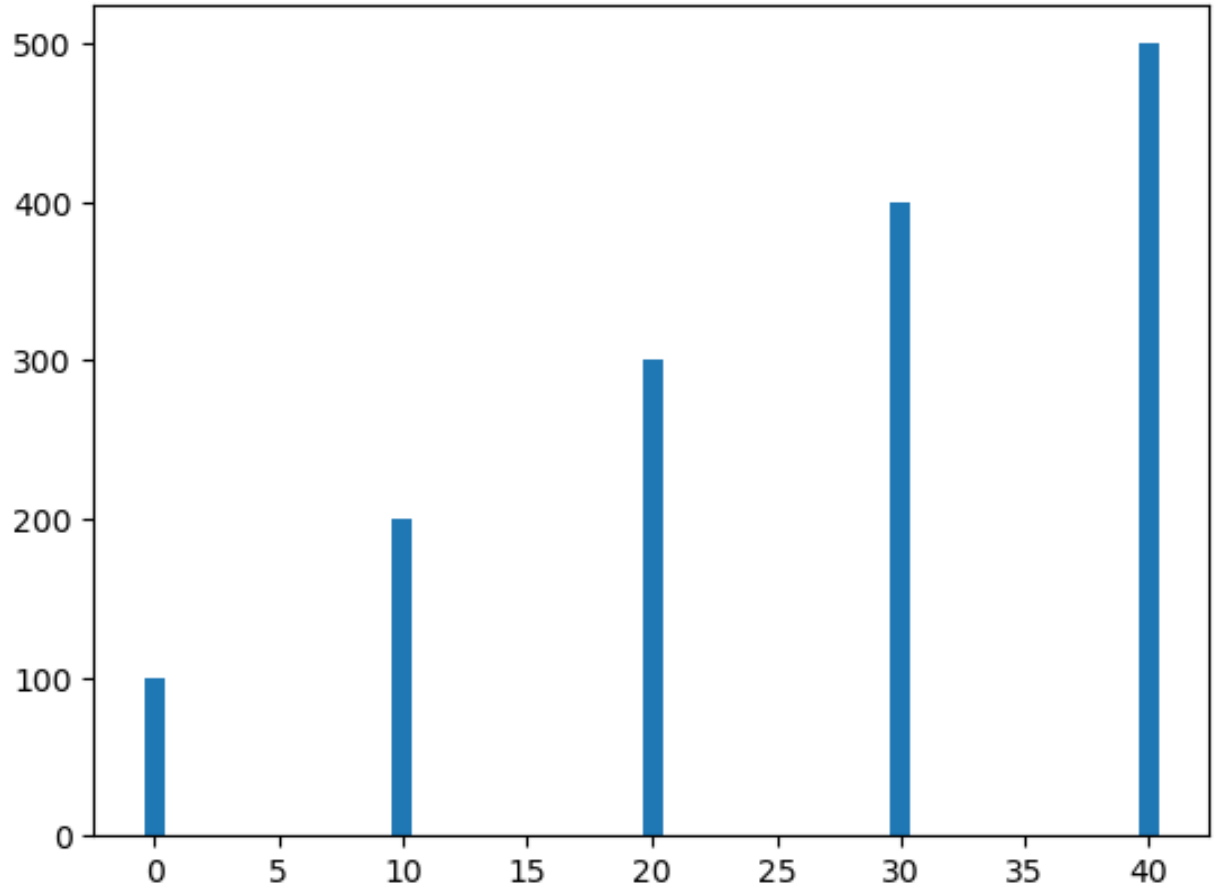
Pie Charts Example – Try it out

•Use a pie chart to represent the **Energy Consumption in a Smart Home** using the data given below:

Components	Energy Consumption in kWh
Smart Lights	120
HVAC	450
Smart Plugs	200
Security Cameras	80
Others	50

Bar Charts Example

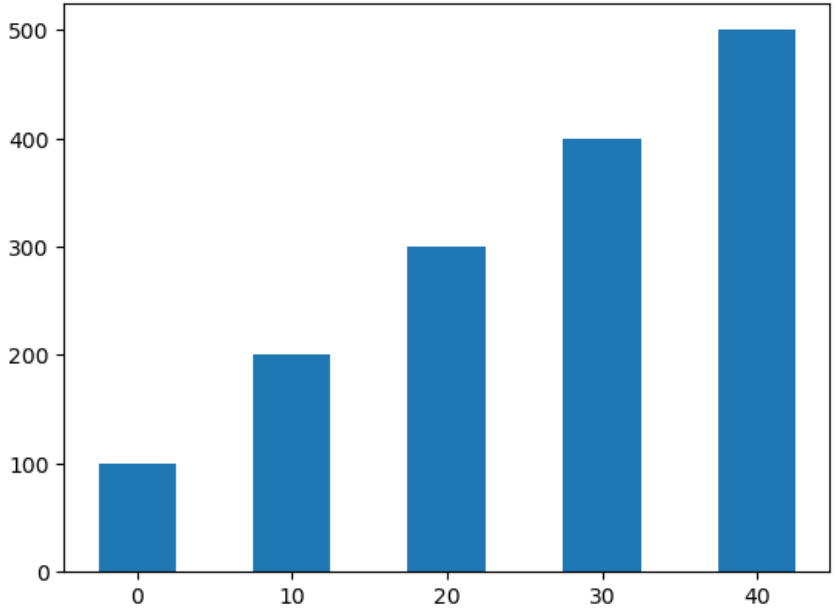
```
import matplotlib.pyplot as plt
left_edges = [0, 10, 20, 30, 40]
heights = [100, 200, 300, 400, 500]
plt.bar(left_edges, heights)
plt.show()
```



Plotting a Bar Chart

- The default width of each bar in a bar graph is 0.8 along the *X* axis.
- You can change the bar width by passing a third argument to the `bar` function.

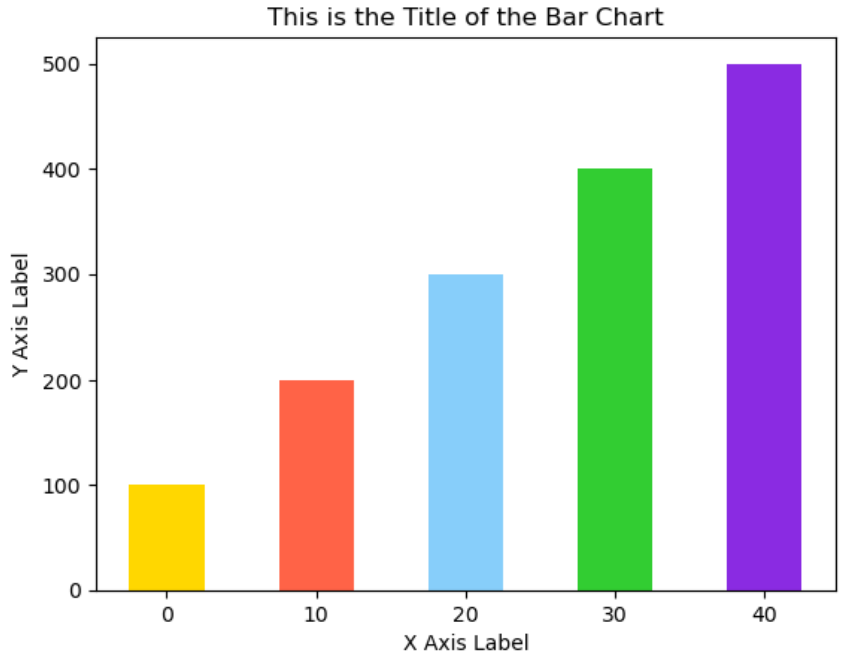
```
import matplotlib.pyplot as plt
left_edges = [0, 10, 20, 30, 40]
heights = [100, 200, 300, 400, 500]
bar_width = 5
plt.bar(left_edges, heights, bar_width)
plt.show()
```



Plotting a Bar Chart

- Use the `xlabel` and `ylabel` functions to add labels to the *X* and *Y* axes.
- The `bar` function has a `color` parameter that you can use to change the colors of the bars.
- The argument that you pass into this parameter is a tuple containing a series of color codes.

```
import matplotlib.pyplot as plt
left_edges = [0, 10, 20, 30, 40]
heights = [100, 200, 300, 400, 500]
bar_width = 5
colors = ['#FFD700', '#FF6347', '#87CEFA', '#32CD32',
'#8A2BE2']
plt.bar(left_edges, heights, bar_width, color=colors)
plt.title("This is the Title of the Bar Chart")
plt.xlabel("X Axis Label")
plt.ylabel("Y Axis Label")
plt.show()
```



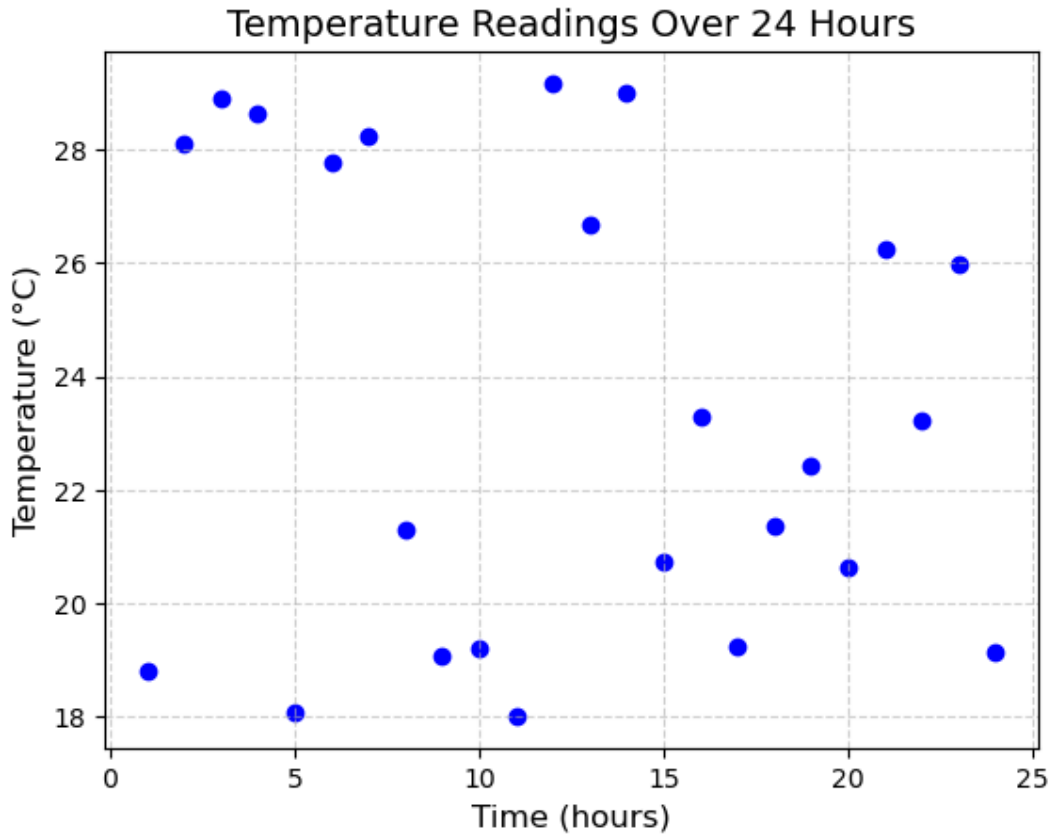
Bar Charts Example – Try it out

•Use a bar chart to represent the **data usage** (in GB) of various IoT devices in a Smart Home using the data given below:

Devices	Data Usage in GB
Smart Lights	5
Security Cameras	50
Smart Thermostat	10
Smart Plugs	8
Smart Door Lock	3

Scatter Plots Example

```
import matplotlib.pyplot as plt
import random

# Generate example data: Time (in hours) and Temperature (in °C)
time = [i for i in range(1, 25)]  # 1 to 24 hours
temperature = [random.uniform(18, 30) for _ in range(24)]
# Random temperatures between 18°C and 30°C

# Create the scatter plot
plt.scatter(time, temperature, color='blue', marker='o')

# Adding titles and labels
plt.title("Temperature Readings Over 24 Hours", fontsize=14)
plt.xlabel("Time (hours)", fontsize=12)
plt.ylabel("Temperature (°C)", fontsize=12)

# Customizing the grid for better readability
plt.grid(True, linestyle='--', alpha=0.6)

# Display the scatter plot
plt.show()
```



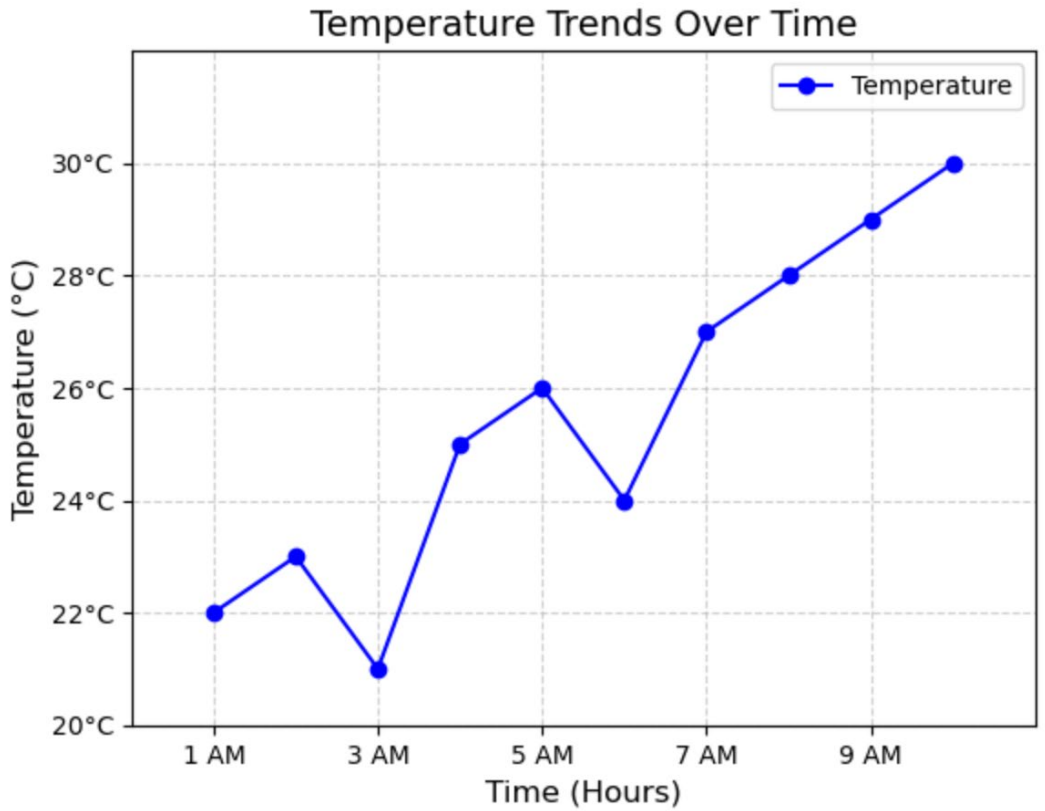
Scatter Plot Example – Try it out

•You are tasked with visualizing the relationship between **temperature** and **humidity** readings collected by an IoT-based weather monitoring system. The system has sensors that capture **temperature (°C)** and **humidity (%)** data in real-time. Your goal is to:

- •Generate 50 random temperature readings between 15°C and 35°C.
- •Generate 50 random humidity readings between 30% and 90%.
- •Plot temperature on the x-axis and humidity on the y-axis.
- •Each point in the scatter plot represents a pair of readings (temperature and corresponding humidity).

Line Plots Example

```
import matplotlib.pyplot as plt
# Example data: Time (in hours) and Temperature (in °C)
time = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  # Time in hours
temperature = [22, 23, 21, 25, 26, 24, 27, 28, 29, 30]  # Temperature readings
# Create the line plot
plt.plot(time, temperature, marker='o', linestyle='-', color='blue', label="Temperature")
# Adding title and labels
plt.title("Temperature Trends Over Time", fontsize=14)
plt.xlabel("Time (Hours)", fontsize=12)
plt.ylabel("Temperature (°C)", fontsize=12)
# Customizing the x-axis and y-axis limits
plt.xlim(0, 11)  # X-axis range from 0 to 11
plt.ylim(20, 32)  # Y-axis range from 20 to 32
# Customizing tick marks
plt.xticks([1, 3, 5, 7, 9], ["1 AM", "3 AM", "5 AM", "7 AM", "9 AM"])
# Custom x-axis tick labels
plt.yticks([20, 22, 24, 26, 28, 30], ["20°C", "22°C", "24°C", "26°C", "28°C", "30°C"])  # Custom y-axis tick labels
# Adding a legend
plt.legend()
# Display the plot
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Line Plots Example - Try it out

•An IoT-based energy monitoring system tracks electricity usage in a smart home at different times of the day. The system collects data every 2 hours for 24 hours, and the goal is to visualize the energy consumption trend throughout the day using a line plot. Your goal is to:

- •Simulate the energy consumption data (in kWh) for 12 time points (2-hour intervals).
- •Use a **line plot** to display the energy consumption trend over 24 hours.
- •Customize the x-axis to show time in a 24-hour format (e.g., "2:00", “14:00").
- •Customize the y-axis to label energy consumption in kWh.
- •Add a legend, gridlines, and appropriate axis labels and title.

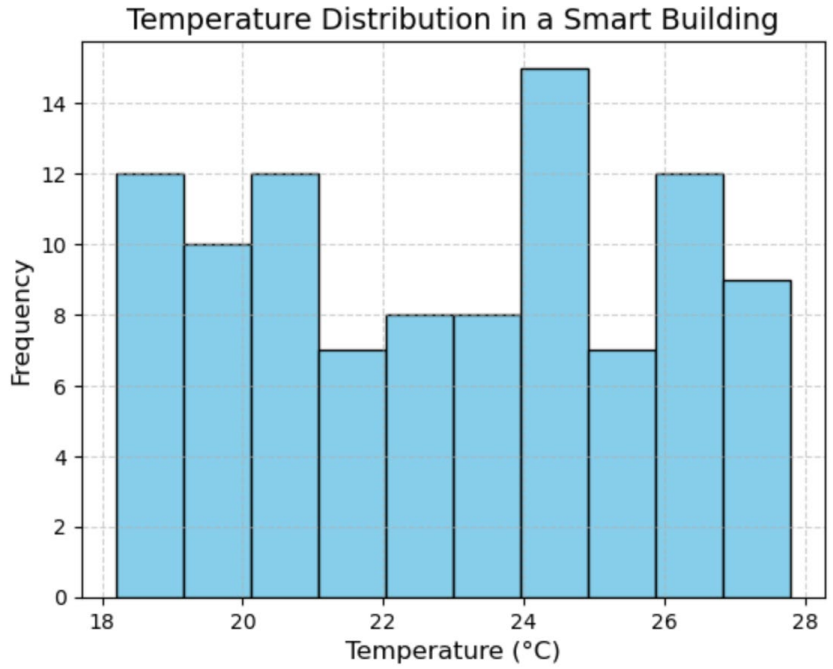
```
time = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22]  # Time in 24-hour format
energy_consumption = [random.uniform(0.5, 2.5) for _ in range(len(time))]
```

Histograms

- Histograms visually represent the relative frequencies of values in a data set using a set of bars.
- Histograms are used to **visualize the distribution of numerical data**.

Histograms Example

```python
import matplotlib.pyplot as plt
import random

# Simulated temperature data (in °C) from IoT sensors
temperatures = [random.uniform(18, 28) for _ in range(100)]
# 100 temperature readings between 18°C and 28°C

# Create the histogram
plt.hist(temperatures, bins=10, color='skyblue', edgecolor='black')

# Adding titles and labels
plt.title("Temperature Distribution in a Smart Building", fontsize=14)
plt.xlabel("Temperature (°C)", fontsize=12)
plt.ylabel("Frequency", fontsize=12)

# Customizing the grid for better readability
plt.grid(True, linestyle='--', alpha=0.6)

# Show the histogram
plt.show()
```



Histograms Example – Try it Out

Scenario: An IoT-based air quality monitoring system collects **PM2.5 (Particulate Matter)** levels from sensors in different areas of a city. PM2.5 levels indicate air pollution, and the data can vary across locations. Your goal is to visualize the distribution of PM2.5 levels recorded by the sensors over a day using a **histogram**. Your goal is:

1. Simulate a dataset of PM2.5 levels collected from 100 IoT sensors in a city.
2. Use a histogram to visualize the frequency distribution of PM2.5 levels.
3. Divide the data into **bins** (e.g., intervals of 10 $\mu g/m^3$).
4. Add appropriate labels, title, and customize the grid and colors.

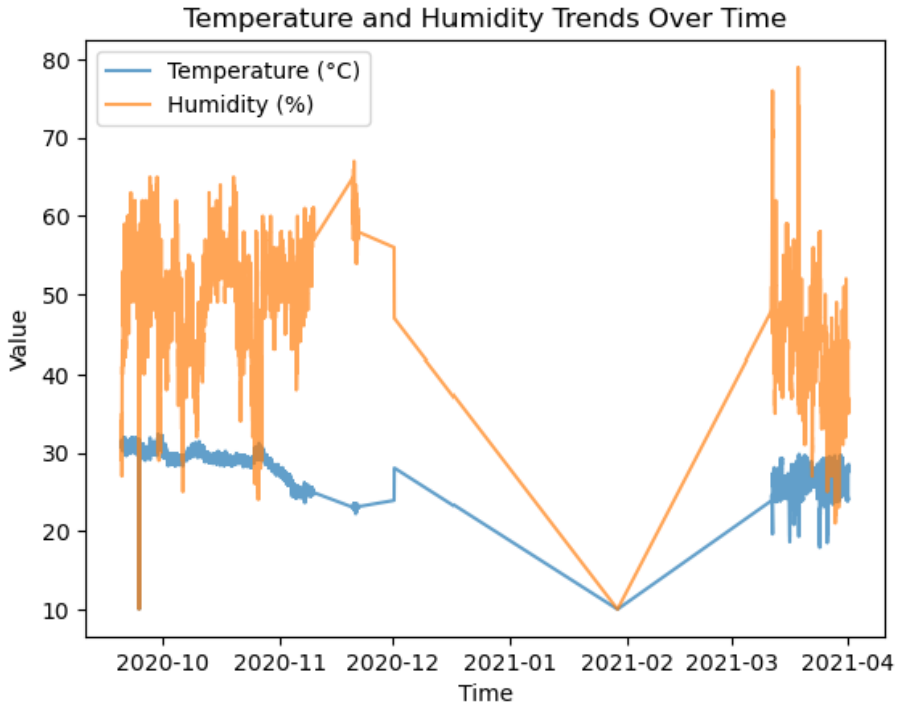
Visualization With Pandas

Analysing Trends Over Time

Visualizing Data with Pandas

- **Objective**: Visualize how variables change over time.
- **Line Plot**: Temperature and Humidity Trends

```
import matplotlib.pyplot as plt

# Convert result_time to datetime
df['result_time'] = pd.to_datetime(df['result_time'])

# Plot temperature and humidity over time
plt.plot(df['result_time'], df['temp'], label='Temperature (°C)', alpha=0.7)
plt.plot(df['result_time'], df['humid'], label='Humidity (%)', alpha=0.7)
plt.title("Temperature and Humidity Trends Over Time")
plt.xlabel("Time")
plt.ylabel("Value")
plt.legend()
plt.show()
```



Introduction to `Seaborn`

- The **`Seaborn`** library is a powerful Python visualization library built on top of `Matplotlib`. It provides a high-level interface for creating attractive and informative statistical graphics with minimal effort.
- **Key Features**:
  - High-level statistical visualizations.
  - Works seamlessly with Pandas DataFrames.
  - Advanced plots like heatmaps, regression plots, and distribution plots.

To use Seaborn, you first need to import it as follows:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

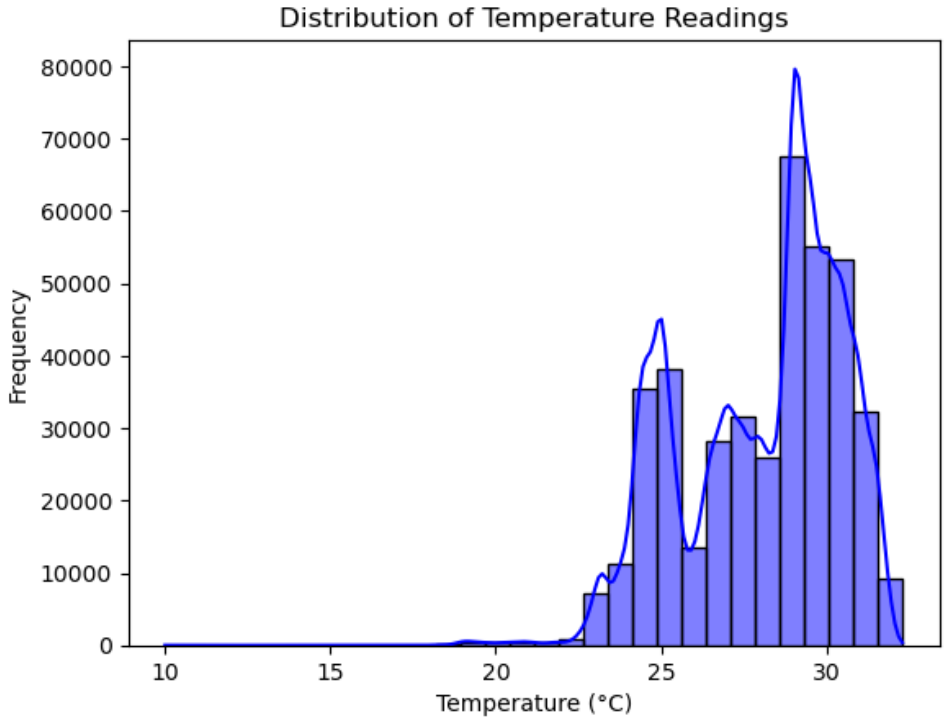
Seaborn provides many functions to create common statistical plots, such as:

- **Histograms**: sns.histplot()
- **Scatter Plots**: sns.scatterplot()
- **Line Plots**: sns.lineplot()
- **Bar Plots**: sns.barplot()
- **Box Plots**: sns.boxplot()
- **Heatmaps**: sns.heatmap()

Understanding Data Distributions

- **Objective**: Explore the distribution of sensor readings.
- **Histogram**: Temperature Distribution

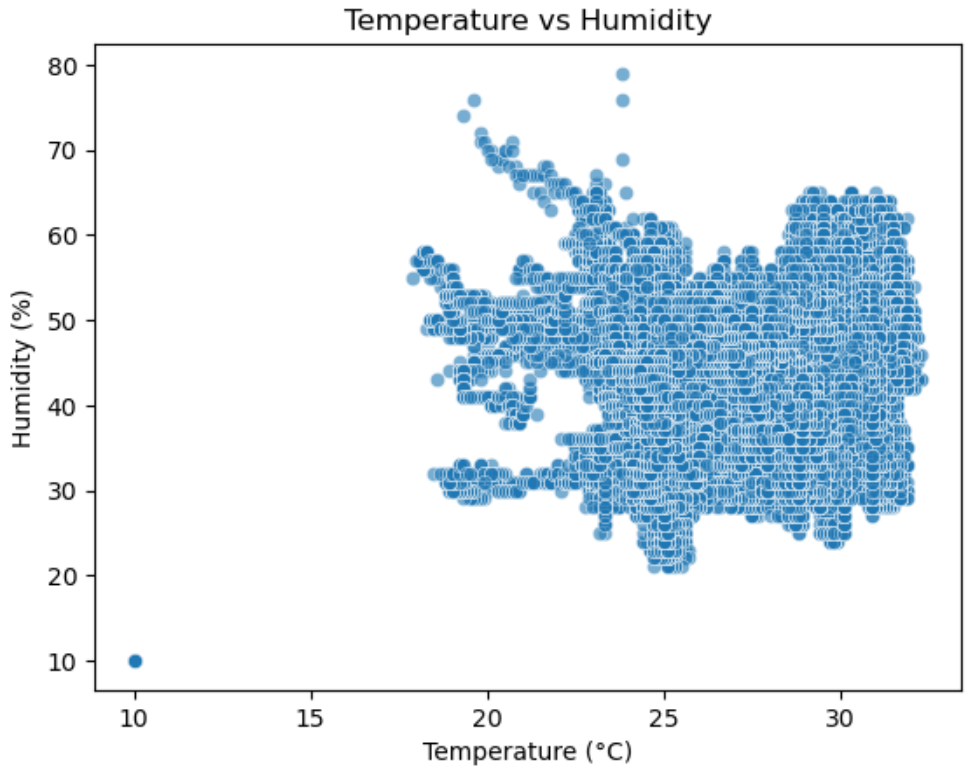
```
import seaborn as sns

# Plot the distribution of temperature
sns.histplot(df['temp'], kde=True, bins=30, color='blue')
plt.title("Distribution of Temperature Readings")
plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")
plt.show()
```



Examining Relationships Between Variable

- **Objective**: Visualize correlations between key variables.
- **Scatter Plot**: Temperature vs. Humidity

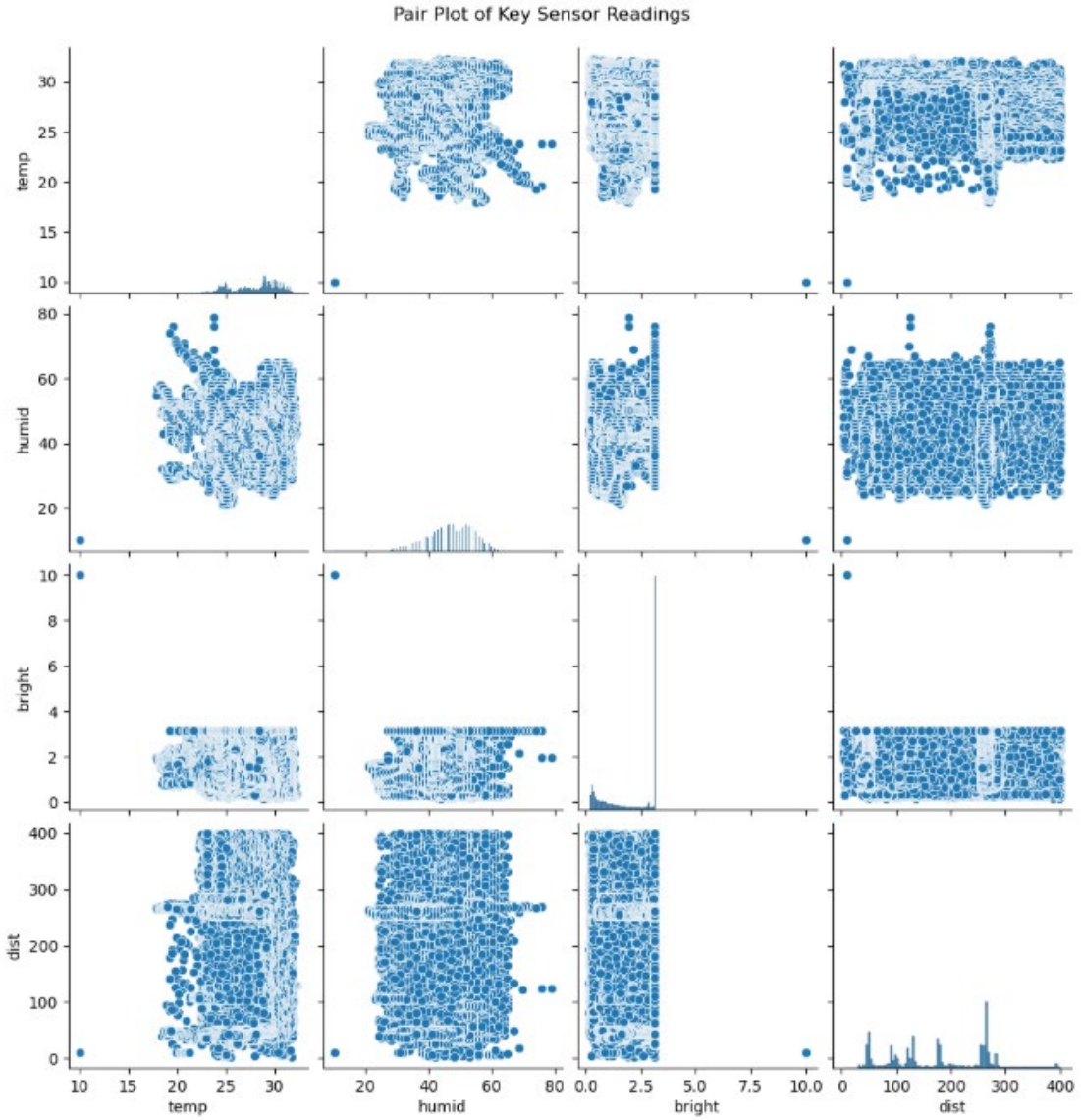
```
# Scatter plot of temperature vs. humidity
sns.scatterplot(x='temp', y='humid', data=df, alpha=0.6)
plt.title("Temperature vs Humidity")
plt.xlabel("Temperature (°C)")
plt.ylabel("Humidity (%)")
plt.show()
```



Examining Relationships Between Variable

- **Pair Plot**: Multi-variable Relationships

```
# Pair plot to explore multiple relationships
sns.pairplot(df[['temp', 'humid',
'bright', 'dist']])
plt.suptitle("Pair Plot of Key Sensor
Readings", y=1.02)
plt.show()
```



Heatmaps for Correlation Analysis

- **Objective**: Analyze how variables are correlated

```
# Compute correlation matrix
correlation_matrix = df[['temp', 'humid',
'bright', 'dist', 'soundlevel']].corr()

# Heatmap of correlations
sns.heatmap(correlation_matrix, annot=True,
cmap="coolwarm", vmin=-1, vmax=1) )
plt.title("Correlation Heatmap of Sensor
Readings")
plt.show()
```